

Nishanth Tharakan, Patrick Lerdsuwanrut, Ajmain Zahin
Chunyang Liao
PIC 16B
12th June 2026

PIC16B Final Project Report

1. Introduction

Pokémon is a Japanese media franchise, created and owned by the company Game Freak, based on cartoonish creatures by the titular name. Specifically in the video games and trading card game, different Pokémon have different stats that affect their use in combat, including numerical attribute statistics (Health, Attack and Defense, Special Attack and Defense, Speed), ability, Typing (19 categories, Pokémon have up to 2), and movesets (up to 4 from a pool specific to that Pokémon).

When encountering a Pokémon in the games, knowing the typing of the Pokémon you are facing can be the difference between dealing enough damage to knock a Water-Ground Pokémon out with a single Grass type move, or dealing no damage with an electric type move. However, knowing the typing of all 1025 Pokémon can be a daunting task, especially when all you are provided is an image of the Pokémon (and its name depending on the game).

Problem Statements and Scientific Questions

As mentioned in the introduction, knowing the typing of pokemon gives you a significant advantage in how the game is played, but doing so manually with limited information is not that easy. With over 1000 pokemon divided into 18 types, even pokemon within the same type can be difficult to always identify correctly based on human intuition alone. However, being a game for kids to play means there should be some amount of intuitiveness that can be used to figure a certain Pokémon's typing. With newer generations introducing more and more complex designs however, this claim needs to be tested, especially due to the fact that a lot of pokemon still remain hard to categorize for new players entering the franchise. Thus, a model which can efficiently identify Pokemon by design not only gives newer players an advantage and easier entry into the game, but can also give us an indication as to how intentional the design choices truly are.

Below we define some of the key problems and questions we want to tackle that motivated the creation of this project:

1. Can a model be trained to identify the typing of a Pokémon solely through images of that Pokémon?
2. How does such a model perform on various subgroups of the data, which includes segregation by actual type, generation released, shiny form, etc?
3. What are the best training regimens to get good results across all Pokémon and their various subgroups?
4. If only given training data for some generations of Pokemons (e.g. only providing data on generations 1-7), how does the model perform on the remaining generations (in this case, 8 and 9)?

Project Objectives/Goals

Our main goals for the project are described in some detail below:

1. Determine feasibility for various models to classify Pokémon sprite images.
2. Figure out which models, methodologies, and training data give the best results in terms of the goal stated above, and how the models perform comparative to our own ability to guess pokemon typing.
3. See what patterns and insights we can gather about pokemon design choices over generations.
4. What can classifying Pokémon sprites tell us about other multi-label classification tasks?

Overview of our Methodology

Below we define our entire project pipeline fully in a few sentences:

1. First we sourced our data from PokeApi and PokeRogue.
2. We then did some exploratory analysis of pokemon type distributions.
3. We then processed the data using the two datasets to create individual sprites. We then used the sprite name to connect them to their types.
4. Then we created some basic models such as Decision Tree, Random Forest, and SVM to see how they perform.
5. We finally decided on using EfficientNet-B0 for our main model and fully implemented it. We also later tried EfficientNet-V2 and Vit-B16 as follow ups, but results were not good enough to justify continuing on.
6. After model implementation, we began analyzing the base metrics.
7. Then we implemented more augmentations, changed parameters, and increased sample size to evaluate performance.

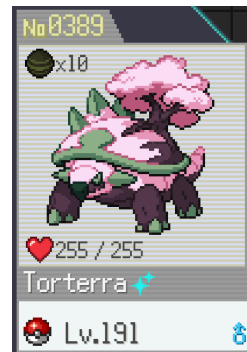
8. Finally, we tried things like greyscaling, feature visualization, and generation-wise sample splitting to see what we could learn about model performance and also pokemon designs in general.

2. Data Collection and Processing

We had to compile two separate datasets to create our training/testing data:

1. Pokeroque has fan made sprites for all 9 generations of Pokemon (except for *Pokémon: Legends Z-A* Mega Evolutions, which will not be tested in this dataset) in a consistent pixel art format, which contrasts Game Freak's change to 3d models for their Pokémon sprites since Generation 6 in 2013). While we could have used official Pokémon 3d models from Pokémon Home, using them would present computational difficulties for us as they are very high quality models.

On right: Example of Pokeroque sprite, with a special Pokeroque-exclusive color palette. For our work, we are only using the default color pallets for all Pokémon.



2. PokeAPI was our source for form and typing information for all Pokémon we are testing. It's a RESTful API containing almost all historical information on every Pokémon, but we will be using only the typing information they provide.

On right: PokeAPI logo



We used the *requests* Python library to get PokeAPI data, and a shell script to pull the Pokeroque Github Repository, isolate the sprites folder, and delete the rest. (We didn't use Python because we were unsure how to do git pulls and large deletions with Python)

Once the sprite sheets were collected, code was implemented to go through them all and split them into sheets based on individual pokemon. Then, the code would crop individual sprites from the sheet into separate PNGs for the model to use.

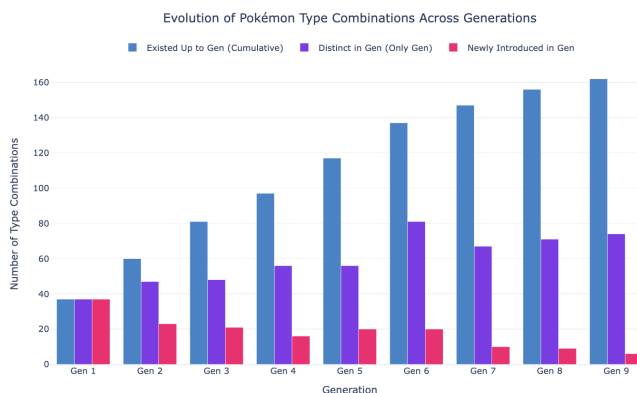
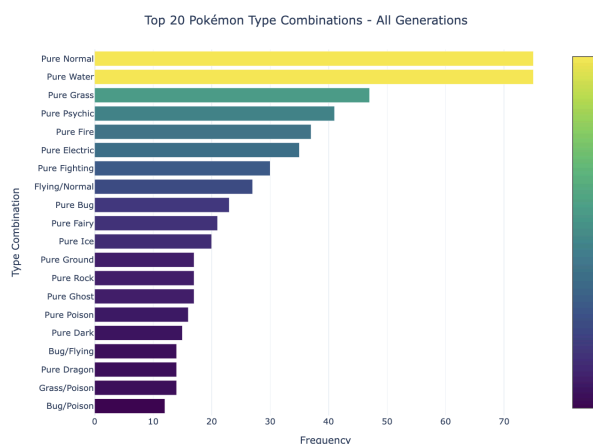
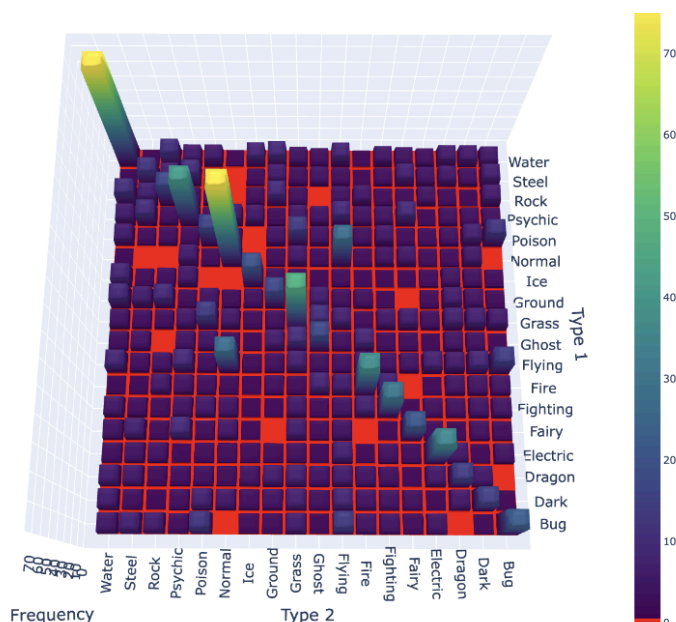
Initial Analysis

There are $18 + \binom{18}{2} = 171$ distinct

Pokemon Types, meaning on average, each typing will have $1024/171 \approx 6$

Representatives. Even accounting for the 9 unused type combinations and the forms not accounted for by the pokedex number (both Raticate and Alolan Ratican have the same Pokedex number), we can see that the representation for each type is not particularly good. At the same time, some type combinations, like Water, Flying, and Normal-Flying, have disproportionately high amounts of representation in the data set, with over 20 representatives each.

With this data, we can conclude that it is probably not feasible to have a model identify both types for a given sprite, but we continue the project by also rewarding the model just as much for a partial identification (such as identifying a Water-Flying type as a Water-Grass or mono-Flying type)



Top Right: 3d heat map of all 171 type combinations as a symmetric matrix. Bright colors means more representatives, red means no representatives.

Bottom Left: Bar graph of the top 20 type combinations, single types identified as "Pure," dual types identified with a slash

Bottom Right: the progression of Pokemon types by generation. Almost every generation introduced 100-200 new pokemon, but there is a clear trend of later generations making those pokemon have more distinct typings.

3. Models

Non-Deep Learning Baselines

Three flat-feature classifiers were built to establish lower bounds and show where deep learning is actually needed. All three use the same pipeline:



- Decision Tree learns if/else rules from PCA components. Fast and interpretable, but it overfits easily and misses complex visual patterns.
- Random Forest trains 100 trees on random subsets of data and features, then takes a majority vote. More robust than a single tree, but still limited by the flat pixel representation.
- SVM (RBF kernel) finds a max-margin hyperplane in PCA feature space. The RBF kernel allows non-linear boundaries, making it the strongest of the three baselines.
-

Nevertheless, PCA-compressed pixels are poor features for sprite classification regardless. All three are single-label classifiers. They cannot predict a secondary type at all, which will not help their scoring.

The deeper problem is the flattening step itself. Pixel (0,0) and pixel (63,63) become unrelated features the moment you flatten, and all spatial structure is gone. PCA then compresses 12,288 dimensions down to 50, keeping only the broadest color gradients. The shape and texture that actually separates a Fire-type sprite from an Electric-type disappears entirely.

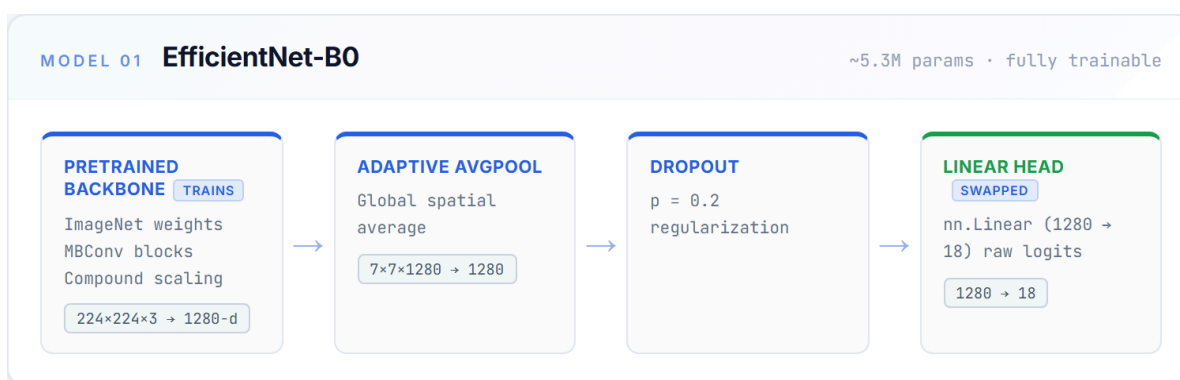
Deep Learning Model Architectures

EfficientNet-B0

We built the initial EfficientNet-B0 pipeline, which became the foundation for the full Classification/ folder. EfficientNet-B0 is a Convolutional Neural Network (CNN) pretrained on ImageNet. The only change from stock EfficientNet was swapping the final layer to output 18 type scores instead of 1000 ImageNet classes:

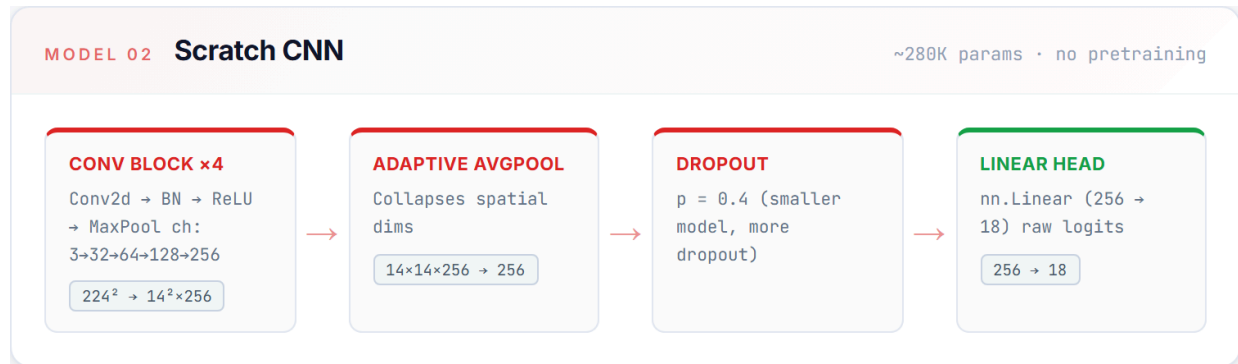
```
`model.classifier[1] = nn.Linear(model.classifier[1].in_features, 18)`
```

Everything before that layer keeps its ImageNet weights and trains end-to-end. The model already knows how to detect edges and color patterns before seeing a single Pokemon sprite.



Softmax vs. Sigmoid: We initially proposed softmax for the output layer. The switch to sigmoid was what actually made dual-type prediction work. Softmax forces all 18 type probabilities to sum to 1, so predicting Fire for Charizard would actively suppress Flying. Sigmoid scores each type independently, so Fire and Flying can both score high at the same time. The loss function changes to BCEWithLogitsLoss: 18 separate binary decisions summed per sample. Raising this question is what got us to the right formulation.

Scratch CNN



EfficientNet-B0 was originally trained for identifying real life animals and objects, but we are instead using it to identify cartoon-style characters. To quantify the usefulness of this pretrained model, we also are using an untrained CNN as a competitor, which otherwise works identically to EfficientNet-B0.

ViT-B/16

ajz434@g.ucla.edu ajmainzahin2005@gmail.com



4. Training Regimen

For training we followed these steps:

1. Dataset Split & Setup:

Generates the split indices, ensuring all sprites for a specific Pokémon ID stay together

```
`dataset = PokemonSpriteDataset()
```

```
# Split strategy selection
```

```
if args.split == "stratified":
```

```
    train_idx, val_idx, test_idx = gen_stratified_split(
```

```
        dataset.index, val_frac=args.val_frac, test_frac=args.test_frac, seed=args.seed
```

```
    )`
```

2. Model Training Loop

Initializes the model architecture, weights sampler, and trains using Binary Cross Entropy loss:

```
`# Create loader & build model
sampler = make_weighted_sampler(dataset, train_idx)
train_loader = DataLoader(Subset(dataset, train_idx), batch_size=args.batch_size,
sampler=sampler)
model = build_model(num_classes=len(TYPES),
freeze_backbone=args.freeze_backbone).to(device)

# Epoch loop with hotswapping transforms
for epoch in range(1, epochs + 1):
    dataset.transform = active_train_tf
    train_m = run_epoch2(model, train_loader, criterion, optimizer, device, train=True)

    dataset.transform = active_eval_tf
    val_m = run_epoch2(model, val_loader, criterion, optimizer, device, train=False)
```

3. Checkpoints:

If the F1 score for the validation set improved between epochs, we save the new model:

```
`if val_m["f1"] > best_f1:
    best_f1 = val_m["f1"]
    torch.save({
        "epoch": epoch,
        "model_state": model.state_dict(),
        "val_f1": best_f1,
        "args": vars(args),
    }, checkpoint_path)`
```

4. Inference:

Since the model returns a series of confidence levels, we apply some hard coded logic to determine the single type/dual type quality that the model is telling us:

```
`for i in range(len(probs)):
    sorted_idx = np.argsort(probs[i])[:-1]
    # Guarantee absolute highest guess
    preds[i, sorted_idx[0]] = 1
    # Predict second type if confidence is within GAP_THRESHOLD
    if (probs[i, sorted_idx[0]] - probs[i, sorted_idx[1]]) < 0.25:
        preds[i, sorted_idx[1]] = 1`
```

5. Metrics Printing & Visualization HTML Gallery

Calculates metrics and outputs base64-encoded mistake cards into an interactive HTML report:

Important features:

The dataset exhibits significant class imbalance, characterized by a predominance of Water and Normal types, while types such as Ghost, Dragon, and Ice are underrepresented. Without corrective measures, the model tends to exhibit bias toward the majority classes. To mitigate this, a weighted sampler was implemented. Each training sample is assigned a weight inversely proportional to the frequency of its rarest type. Consequently, samples from underrepresented classes appear more frequently in training batches, ensuring consistent exposure to minority class features.

Augmentation

[Section TBD]

Grayscale Ablation

[Section TBD]

Inference — Gap Threshold

Inference Gap Threshold Design

Originally, we preset the amount of labels the model would label an image with; when testing, we would tell the model if it was one or two types, and we would take the top 1 or 2 accordingly. However, this makes the fatal assumption that we would already know how many types a Pokemon would have, which would defeat the point of letting the model figure it out. Instead, we have preset a difference threshold, which utilizes the assumption that if the model thinks a Pokémon has more than one type, those secondary types would have similar confidence values to the primary type it wants to label the image with. As such, if the model's second option's confidence value is within 25% of the first type's, then the image is classified as a dual type:

1. Always lock in the absolute #1 highest guess

```
preds[i, sorted_idx[0]] = 1
```

```
# 2. Check if the 2nd highest guess is within the gap threshold
if (probs[i, sorted_idx[0]] - probs[i, sorted_idx[1]]) < GAP_THRESHOLD:
    preds[i, sorted_idx[1]] = 1`
```

This is how the model actually runs without knowing the answer in advance.

Evaluation Metrics ajmainzahin2005@gmail.com

We reported F1, Precision, and Recall instead of just accuracy. On an imbalanced multi-label problem, accuracy alone is a bad proxy. A model that always predicts water can score well on accuracy while having learned nothing. Macro F1 averages across all 18 types equally, so rare types like Ghost actually affect the score.

Custom overlap metrics we added:

At Least 1 Right: the model predicted at least one correct type

2 Types Right (Dual): among dual-type Pokemon specifically, both types correct

Per-type breakdown: F1/Precision/Recall for each of the 18 types

Per-generation breakdown: accuracy by generation, as a check for distribution shift

The generation breakdown tests whether the model learned type-indicative features or just visual patterns specific to older sprite generations.

5. Results Analysis

- S

6. Conclusion

[Section TBD]

7. Member Contributions

- Patrick:
 - Developed the flat-feature classifiers (Decision Tree, Random Forest, SVM) and the initial EfficientNet-B0 pipeline; designed the weighted sampler and co-designed the inference gap threshold;
- Nishanth:
 - Data acquisition: wrote all scripts related to getting Pokeroque and PokeAPI data (Patrick modified them to allow multithreading)
 - Suggested the EfficientNet-B0 model; implemented the Scratch CNN.
- Ajmain: Implemented the ViT-B/16 architecture; implemented the inference gap threshold, selected and reported evaluation metrics.

8. References:

[PokéRogue](#)

[PokeAPI](#)